FinMate: A Personal Financial Decision Support System Using Multi Agent Intelligence and Retrieval Augmented Generation

First Author, Second Author, Third Author, Fourth Author, Fifth Author
Department of Computer Science and Engineering
Institution Name
City, Country
email@example.com

***Abstract*—Personal financial management requires users to interpret transaction data, monitor expenditure, evaluate financial goals, and make informed decisions under uncertainty. This paper presents FinMate, a personal financial decision support system that combines deterministic financial computation, machine learning, retrieval augmented generation, multi agent orchestration, and privacy focused mobile transaction ingestion. The system uses a FastAPI based backend with specialized agents for transaction analysis, budgeting, financial goals, investments, and decision synthesis. Transaction classification uses a TF-IDF and Linear Support Vector Classifier pipeline, expense forecasting uses Ridge Regression with lagged temporal features, and expenditure anomalies are detected using Isolation Forest. A regulatory retrieval pipeline provides context from RBI, SEBI, and FPSB India guidance. Monetary computations are separated from language model generation using Python Decimal arithmetic. State-changing operations are controlled through human-in-the-loop approval, while the Android ingestion pipeline parses authorized transaction SMS messages locally without transmitting raw SMS content. The implemented system reports 98.4 percent transaction classification accuracy, a macro F1 score of 0.9849, a 93.3 percent retrieval Hit Rate@3, and 102 out of 102 automated tests passing.**

***Index Terms*—personal finance, financial decision support, machine learning, retrieval-augmented generation, multi-agent systems, large language models, financial data privacy, anomaly detection**

I. Introduction

Personal financial management involves a combination of transaction tracking, expenditure analysis, budgeting, savings planning, goal management, and investment related decision support. The increasing availability of digital financial records has created opportunities for automated analysis, but the usefulness of such systems depends on their ability to transform heterogeneous financial data into accurate, interpretable, and actionable information.

Conventional personal finance applications commonly emphasize transaction recording, dashboards, categorization, and historical summaries. Such functionality is useful for understanding past financial activity, but it does not necessarily provide an integrated mechanism for interpreting financial scenarios, comparing alternatives, forecasting expenditure, or grounding recommendations in authoritative financial guidance. At the same time, large language models (LLMs) provide natural language interaction and reasoning capabilities, but their direct use for financial computation introduces risks because generated numerical values may not correspond to authoritative transaction records or deterministic financial calculations.

FinMate addresses this problem through a layered financial decision support architecture that separates numerical computation from natural language generation. The system combines a deterministic financial calculation engine with machine learning models, retrieval augmented generation (RAG), and a collaborative multi agent orchestration layer. The architecture is implemented using Python, FastAPI, Pydantic, SQLAlchemy, PostgreSQL, and Scikit-learn, with a React based web workspace and a native Android client for privacy focused transaction ingestion.

The proposed system employs specialized financial agents for transaction analysis, budgeting, goal management, investment related educational analysis, and cross-domain decision synthesis. An orchestrator performs intent triage and controls agent execution through bounded tool access. Financial values used by the language model are obtained from backend calculation services rather than generated directly by the model. This separation is intended to reduce numerical inconsistencies and improve the traceability of financial responses.

FinMate also incorporates retrieval augmented generation for financial regulatory and planning knowledge. The implemented retrieval subsystem uses vector representations and cosine similarity to retrieve relevant content from a knowledge corpus covering RBI, SEBI, and FPSB India guidelines. Retrieved information can subsequently be supplied as context for natural language explanations.

Privacy is addressed at the transaction ingestion layer through the native Android FinMateSMS component. Bank transaction alerts are filtered and parsed locally using deterministic regular expressions. The system uses sender whitelisting, OTP and secret-PIN filtering, local hashing, and a Room based transaction queue before synchronized transaction data is sent to the backend. Raw SMS content is not transmitted by the ingestion pipeline.

A further design objective is to maintain human control over operations that could modify financial records. FinMate classifies operations into read-only actions, simulations, and state-mutating actions. Mutating operations are staged through a human-in-the-loop approval mechanism and require explicit user confirmation before persistent state is modified.

The principal contributions of this work are as follows:

- A unified personal financial decision support architecture combining deterministic financial computation, machine learning, retrieval augmented generation, and multi agent orchestration.
- A separation between numerical financial computation and language model generation using exact decimal arithmetic and structured backend outputs.
- A governed multi agent architecture with specialized financial agents, constrained tool access, execution limits, and human approval for state changing operations.
- A regulatory retrieval pipeline based on semantic similarity over a financial knowledge corpus containing RBI, SEBI, and FPSB India guidance.
- A privacy focused Android transaction ingestion mechanism that performs SMS parsing locally and avoids transmission of raw SMS content.
- An implemented evaluation framework covering machine learning performance, retrieval performance, agent behavior, application functionality, and automated testing.

The remainder of this paper is organized as follows. Section II discusses related work and identifies the system gap addressed by FinMate. Section III defines the problem and design objectives. Section IV presents the proposed system architecture. Section V describes the methodology and implementation. Section VI discusses the machine learning, retrieval augmented generation, and language model components. Section VII presents the security and privacy architecture. Section VIII evaluates the implemented system. Section IX discusses the findings and limitations, followed by future work in Section X and the conclusion in Section XI.

II. Related Work

Artificial intelligence has increasingly been applied to personal financial management to support budgeting, expenditure analysis, forecasting, and decision making. Vaduka *et al.* studied AI-driven decision support for personal finance and identified applications across budgeting, investment, debt management, and retirement planning while also noting privacy and transparency as important deployment concerns [1]. Shailendra *et al.* presented Money Map, a data-driven personal finance tracker that combines machine learning, budget forecasting, visualization, and personalized financial insights [2]. Lakshmi *et al.* described an AI-powered financial consultant incorporating budget management, expense tracking, financial education, financial calculators, and predictive models for financial analysis [3]. These systems demonstrate the feasibility of AI-assisted personal finance, but they primarily emphasize integrated application functionality rather than the combination of deterministic financial computation, governed multi-agent execution, regulatory retrieval, and privacy-focused transaction ingestion considered in FinMate.

Transaction categorization is an established application of machine learning in financial software. Recent work has investigated automated classification of personal expenses from unstructured transaction descriptions using natural language processing and machine learning methods [4]. Related research has also explored transaction anomaly detection using supervised and unsupervised machine learning methods for identifying deviations in financial transactions [5], [6]. These studies support the use of machine learning for transaction-level analysis. FinMate extends this task within a broader decision-support pipeline in which classification and anomaly signals become inputs to downstream financial analysis rather than isolated prediction outputs.

Forecasting and predictive analytics are also increasingly used in personal financial applications. Recent research has investigated automated transaction classification together with budget prediction, showing the growing use of predictive models for personal financial planning [7]. FinMate similarly performs expenditure forecasting, but uses a strictly lagged feature construction strategy and separates forecast computation from language model narration. This separation allows financial values and uncertainty intervals to originate from backend model outputs rather than being generated directly by a language model.

Large language models have introduced natural language interfaces for complex information and decision-support tasks. However, standalone language models can suffer from factual hallucination and limited access to current or domain-specific knowledge. Retrieval-Augmented Generation (RAG) addresses these limitations by combining language generation with retrieval from an external knowledge source [8]. Recent surveys further describe the evolution of RAG toward more advanced and modular retrieval pipelines and emphasize the importance of grounding generated responses in verifiable external information [9]. These properties are particularly relevant to financial applications where regulatory guidance and other authoritative information should be distinguishable from generated explanations.

Multi-agent architectures provide another mechanism for decomposing complex tasks into specialized components. Surveys of multi-agent systems describe architectures in which agents coordinate through structured communication and workflow mechanisms [10]. More recent work on large model-based agents highlights planning, tool use, cooperation, security, and privacy as important considerations when building agentic systems [11]. A 2026 survey of LLM agents similarly identifies memory, planning, and tool use as central components while highlighting reliability and evaluation challenges [12]. FinMate applies these concepts to personal financial decision support through domain-specific agents, a centralized orchestrator, bounded tool access, execution limits, and explicit human-in-the-loop approval for persistent state changes.

The literature therefore establishes several relevant capa-

bilities: AI-based personal financial management, machine learning for transaction analysis, RAG for knowledge grounding, and multi-agent architectures for task decomposition. However, these capabilities are commonly studied as separate components. FinMate combines them in a single decision-support architecture and adds two implementation constraints that are central to the proposed system: monetary computations are performed by a deterministic financial engine rather than generated by the language model, and state-changing operations are subject to explicit human authorization. The Android ingestion component additionally performs transaction extraction locally from authorized bank SMS messages, supporting a privacy-oriented data collection workflow.

The resulting research gap is therefore not the invention of an individual machine learning model, RAG, or multi-agent architecture. Rather, the focus is the engineering and evaluation of their coordinated use for personal financial decision support under deterministic computation, regulatory grounding, privacy constraints, and human governance.

III. Problem Definition and Design Objectives

The objective of FinMate is to provide an intelligent financial decision support system capable of transforming heterogeneous personal financial records into transparent analytical information and scenario based decision support while maintaining deterministic numerical correctness and human control.

The system addresses the following requirements:

1) Accurate classification and organization of transaction data.
2) Forecasting of future expenditure from historical financial patterns.
3) Identification of anomalous expenditure behavior.
4) Retrieval of relevant financial and regulatory guidance.
5) Natural language interaction through large language models.
6) Deterministic computation of monetary values independent of language model generation.
7) Simulation of financial scenarios without modifying persistent records.
8) Human authorization for state changing operations.
9) Privacy preserving ingestion of transaction information.
10) Reliable storage, validation, lineage tracking, and duplicate prevention for financial records.

The resulting system is designed as a decision support platform rather than an autonomous financial agent. It does not independently execute trades, transfer funds, or modify financial records without verified human authorization.

IV. Proposed System Architecture

FinMate is a layered personal financial decision-support system that integrates client applications, a REST API gateway, multi-agent orchestration, machine learning, regulatory retrieval, deterministic financial computation, data-quality services, and persistent storage. The architecture separates financial computation from language generation and places explicit controls around operations that can modify persistent financial state.

A. System Overview

The system provides two primary client interfaces: a React-based web workspace and a native Android client named FinMateSMS. The web workspace provides financial dashboards, transaction management, AI-assisted analysis, scenario simulation, and human-in-the-loop review. FinMateSMS provides privacy-focused transaction ingestion by processing authorized bank SMS alerts locally.

Both clients communicate with a FastAPI REST gateway. The backend exposes versioned endpoints under `/api/v1` and uses Pydantic v2 for request and response validation. Domain services handle transactions, financial goals, investments, data ingestion, machine learning, retrieval, and multi-agent execution.

Persistent data is managed through SQLAlchemy 2.0 with PostgreSQL 16 as the primary database. SQLite is supported as a fallback, and Alembic manages database migrations.

B. Client Layer

The web workspace is implemented using React 19, TypeScript, Vite, Tailwind CSS v4, Lucide React, and Recharts. It provides the primary interface for financial analysis and includes an AI Advisor Hub, financial dashboard, transaction ledger, CSV ingestion workflow, goals and portfolio views, scenario analysis, and human-in-the-loop review.

The native Android client uses Kotlin 1.9+ and Android SDK APIs 24–34. AndroidX Room provides local persistence, while WorkManager manages background synchronization. Retrofit and OkHttp provide communication with the backend. Transaction alerts are parsed locally and only structured transaction information is synchronized.

C. FastAPI REST Gateway

The FastAPI gateway provides the communication boundary between clients and backend services. Its responsibilities include request validation, routing, authentication context handling, and access to domain services.

The API supports multi-agent task execution, human approval workflows, transaction analysis, forecasting, anomaly detection, regulatory retrieval, SMS-based ingestion, CSV processing, financial goals, investments, and decision simulation. This separation keeps presentation logic independent from financial computation and AI services.

D. Multi-Agent Orchestration

FinMate uses a centralized AI Orchestrator to route financial queries to specialized agents. The principal agents are:

- **Transaction Specialist**: analyzes transactions, categories, spending patterns, and anomalies.
- **Budget Specialist**: evaluates expenditure, surplus, savings rates, and projected spending.
- **Goal Specialist**: evaluates financial goals and required contribution rates.

- **Investment Specialist**: provides investment-related educational and allocation analysis with appropriate safeguards.
- **Financial Decision Agent**: synthesizes evidence from multiple specialists into structured decision alternatives.

The orchestrator performs intent triage and coordinates specialist execution. Agent execution is bounded by a maximum of eight agent steps, twelve tool calls, and three agent handoffs.

E. Governed Tool Registry

FinMate contains a governed registry of fifteen domain tools. Each tool uses structured input and output contracts enforced through Pydantic schemas. Agent-specific allowlists restrict access to tools required for each domain.

The registry also applies tenant context to domain operations. Thus, agent execution is constrained both by the tools an agent can invoke and by the financial context in which those tools operate. This provides a least-privilege boundary between language-driven orchestration and persistent financial data.

F. Analytical Services

The analytical layer contains three primary machine learning services:

- transaction classification using TF-IDF and LinearSVC;
- expenditure forecasting using Ridge Regression with lagged temporal features; and
- anomaly detection using Isolation Forest.

These services return structured analytical outputs rather than natural language responses.

FinMate also provides a regulatory retrieval service based on TF-IDF representations and cosine similarity. Its knowledge corpus contains guidance associated with RBI, SEBI, and FPSB India. Retrieved entries are provided as contextual evidence to the language-model layer.

G. Deterministic Financial Computation

A dedicated financial computation layer performs authoritative monetary calculations independently of the language model. Python's `Decimal` type with `ROUND_HALF_UP` is used for financial arithmetic, while persistent monetary values use SQL `NUMERIC(15,2)` fields.

The computation layer provides calculations including monthly surplus, savings rate, goal contributions, financial runway, and scenario changes. Consequently, the LLM does not serve as the source of authoritative monetary values.

H. Counterfactual Simulation

FinMate provides an in-memory what-if simulation sandbox. Potential changes such as purchases, savings adjustments, expense changes, goal contributions, debt payments, investments, subscriptions, and income changes can be evaluated without modifying persistent records.

The simulation produces temporary financial states and compares effects on surplus, savings, financial buffers, and goal timelines. Since the simulation is isolated from the persistent ledger, exploratory scenarios do not directly modify stored financial records.

I. Data Engineering and Persistence

The ingestion subsystem supports heterogeneous CSV files by detecting common delimiters and supported encodings. A normalization layer handles different date formats, credit/debit representations, and currency symbols. An eleven-rule quality engine classifies records into error, warning, and informational categories.

Duplicate detection is performed using transaction attributes for structured imports, while Android transactions use SHA-256 source references with local and server-side duplicate checks. Source type, source reference, import identifier, and quality metadata are retained for lineage.

Core persistent entities include users, financial profiles, transactions, categories, goals, investments, import records, agent tasks, approvals, and tool calls.

J. End-to-End Processing Flow

The principal processing flow is:

1) A user submits a financial query or transaction through the web or Android client.
2) The FastAPI gateway validates the request.
3) The orchestrator identifies the financial domain and selects the required specialist agents.
4) Agents obtain structured evidence through permitted tools and analytical services.
5) Machine learning, deterministic computation, database services, and regulatory retrieval provide supporting evidence.
6) The Financial Decision Agent synthesizes the evidence into structured alternatives and trade-offs.
7) Read-only requests return analytical results, while simulations remain in memory.
8) Operations requiring persistent modification enter the human-in-the-loop approval workflow.
9) Persistent state is changed only after explicit user approval.

This architecture establishes clear boundaries between data acquisition, financial computation, analytical models, knowledge retrieval, language generation, and persistent state mutation.

V. Methodology and Implementation

FinMate follows a modular implementation methodology in which financial data acquisition, validation, deterministic computation, machine learning, retrieval, agent orchestration, and decision presentation operate as separate but coordinated services. The design places deterministic financial operations outside the language-model generation process.

A. Financial Data Acquisition

FinMate supports structured CSV transaction imports and transaction ingestion through the native Android FinMateSMS client.

The CSV pipeline identifies common delimiters and supported encodings and normalizes heterogeneous date formats, credit/debit conventions, and currency representations into the internal transaction schema. Imported records retain source metadata and are associated with an import identifier for subsequent traceability.

For Android ingestion, authorized bank SMS messages are filtered and parsed locally. The resulting structured transaction is stored in the Android Room database before synchronization with the backend.

B. Data Quality and Validation

Before records enter the persistent financial ledger, FinMate applies an eleven-rule deterministic data-quality engine. Rules are classified into three levels:

- **Error**: identifies records that cannot be safely processed, such as invalid dates, amounts, or required fields.
- **Warning**: identifies potentially problematic records, including possible duplicates, anomalies, or unclassified transactions.
- **Information**: records successful normalization and enrichment operations.

Records containing errors are prevented from entering persistent storage, while warnings are retained as metadata for downstream analysis.

C. Deduplication and Lineage

Structured imports use a sliding-window comparison involving user identity, transaction date, amount, and normalized description to identify potential duplicates.

For Android transactions, a SHA-256 hash is generated from the processed SMS information and used as a stable source reference. Android Room enforces local uniqueness, while the backend performs an additional duplicate check during synchronization.

The duplicate condition can be represented as

$$D = \begin{cases} 1, & h \in H_{\text{local}} \vee h \in H_{\text{server}}, \\ 0, & \text{otherwise}, \end{cases} \tag{1}$$

where h is the transaction hash and the two sets contain previously recorded local and server-side references.

Transaction lineage is maintained through source type, source reference, import identifier, and data-quality metadata.

D. Deterministic Monetary Computation

FinMate uses Python's `Decimal` type with `ROUND_HALF_UP` for monetary calculations. Persistent monetary values use SQL `NUMERIC(15,2)`.

Monthly surplus is calculated as

$$S = I - (F + D), \tag{2}$$

where I represents monthly income, F represents fixed expenditure, and D represents discretionary expenditure.

The savings rate is calculated as

$$R_s = \frac{C_s}{I_t} \times 100, \tag{3}$$

where C_s represents savings contributions and I_t represents total inflow.

For a financial goal, the required monthly contribution is

$$C_m = \frac{T - C}{M}, \tag{4}$$

where T is the target amount, C is the current accumulated amount, and M is the remaining number of months.

These values are calculated by backend financial services rather than by the language model.

E. Transaction Classification

Transaction descriptions are categorized using a supervised pipeline based on TF-IDF feature extraction and LinearSVC. The classifier maps descriptions to ten financial categories: Housing, Food and Dining, Transportation, Utilities, Healthcare, Entertainment, Shopping, Personal Care, Education, and Investments.

The resulting category is stored as structured transaction metadata and becomes an input to budgeting, anomaly analysis, and downstream decision-support processes.

The implemented system reports 98.4 percent accuracy and a macro F1-score of 0.9849.

F. Expense Forecasting

Future expenditure is estimated using Ridge Regression with strictly lagged temporal features. The feature set includes previous monthly expenditure values, three-month rolling averages, and month information.

For the prediction at time $t+1$, the model can be represented as

$$\hat{y}_{t+1} = f(y_t, y_{t-1}, y_{t-2}, \bar{y}_{t-2:t}, m_{t+1}), \tag{5}$$

where historical expenditure values and the rolling average are derived only from observations available before the prediction point.

TimeSeriesSplit is used during model development to preserve temporal ordering. The implemented system reports a holdout MAE of INR 3,369.93 and produces a point forecast with a 90 percent uncertainty interval.

G. Anomaly Detection

FinMate uses Isolation Forest for unsupervised expenditure anomaly detection with an empirical contamination value of 0.05.

An identified anomaly is treated as an analytical signal rather than as proof of fraud. Unusual expenditure can have legitimate explanations, including exceptional purchases, billing events, duplicate records, or classification errors. Consequently, anomaly signals are passed to the decision-support layer for contextual interpretation.

H. Regulatory Knowledge Retrieval

The regulatory knowledge subsystem contains guidance associated with RBI, SEBI, and FPSB India. Queries and indexed knowledge entries are represented using TF-IDF and compared using cosine similarity.

For vectors a and b,

$$\mathrm{sim}(a, b) = \frac{a \cdot b}{\|a\|\|b\|}. \tag{6}$$

The highest-ranked relevant entries are supplied as contextual evidence to the language-model layer. The implemented retrieval subsystem reports a Hit Rate@3 of 93.3 percent and an average retrieval latency of 0.95 ms.

I. Multi-Agent Decision Processing

The AI Orchestrator performs intent triage and routes a request to the specialized financial agents required for the task. Agent outputs are represented as structured evidence and subsequently synthesized by the Financial Decision Agent.

Agent execution is constrained by maximum steps, tool calls, and handoffs. The resulting workflow allows domain-specific analysis while limiting uncontrolled execution.

J. Counterfactual Decision Simulation

FinMate evaluates financial what-if scenarios using an in-memory representation of the user's financial state. A temporary scenario delta Δ is applied to a baseline state:

$$S_{\mathrm{scenario}} = S_{\mathrm{baseline}} + \Delta. \tag{7}$$

Multiple pathways can be compared using changes in monthly surplus, savings, financial buffer, and goal timelines. Because the simulation does not operate directly on persistent records, exploratory scenarios do not modify the financial ledger.

K. Android SMS Processing

The FinMateSMS client performs transaction extraction locally. Incoming messages are first checked against a financial-institution sender whitelist. OTP and secret-PIN patterns are filtered before transaction parsing.

Eligible messages are processed using deterministic regular expressions to extract structured transaction information. A SHA-256 reference is generated and the transaction is stored in Room. WorkManager subsequently handles synchronization when network connectivity is available.

The backend validates the structured payload, performs duplicate checking, and inserts the transaction when no existing record is found. The raw SMS body is not transmitted.

L. Language Model Integration

FinMate uses a `BaseLLMProvider` abstraction that separates the application from a specific language model provider. The implementation supports `MockLLMProvider`, `OpenAILLMProvider`, and `GeminiLLMProvider`.

The language model receives structured evidence produced by trusted backend services. Depending on the request, this evidence may include financial calculations, forecasts, anomaly signals, scenario results, specialist outputs, and retrieved regulatory information.

Thus, the language model is used primarily for natural-language explanation and synthesis rather than authoritative financial arithmetic.

M. Human-in-the-Loop Governance

FinMate distinguishes read-only, simulation, and mutating operations. Read-only operations can execute without persistent modification. Simulation operations remain in memory. Mutating operations are staged as pending approval records.

A pending operation has a 15-minute expiration window and requires explicit user confirmation before commitment. The resulting state transition is

$$\text{Proposed} \rightarrow \text{Pending Approval} \rightarrow \begin{cases} \text{Committed}, & \text{approved}, \\ \text{Rejected}, & \text{declined or expired}. \end{cases} \tag{8}$$

This design preserves human control over persistent financial state.

N. Implementation Environment

The backend uses Python 3.11+, FastAPI, Pydantic v2, SQLAlchemy 2.0, Alembic, PostgreSQL 16, and SQLite fallback support. Machine learning components use Scikit-learn with NumPy and SciPy.

The web application uses React 19, TypeScript, Vite, Tailwind CSS v4, Lucide React, and Recharts. The Android client uses Kotlin 1.9+, AndroidX Room 2.6.1, WorkManager 2.9.0, Retrofit 2.11.0, and OkHttp 4.12.0. Docker and Docker Compose support containerized deployment.

VI. Artificial Intelligence and Retrieval-Augmented Generation

FinMate combines machine learning, retrieval-augmented generation (RAG), large language models, and multi-agent orchestration within a unified financial intelligence layer. The principal design objective is to ensure that predictive models and deterministic financial services produce structured evidence before natural-language generation occurs.

A. AI Pipeline

The AI subsystem contains three predictive components and one knowledge grounding component:

1) transaction classification;
2) expenditure forecasting;
3) expenditure anomaly detection; and
4) regulatory retrieval followed by language-model synthesis.

The resulting evidence is passed to the decision-support layer rather than allowing the language model to independently generate authoritative financial facts.

B. Transaction Classification

FinMate uses word-level Term Frequency-Inverse Document Frequency (TF-IDF) features followed by a Linear Support Vector Classifier (LinearSVC) for transaction classification.

The classifier maps transaction descriptions to ten predefined financial categories. The implemented model reports 98.4 percent classification accuracy and a macro F1-score of 0.9849.

The classification result is retained as structured transaction metadata and can subsequently be consumed by budgeting, anomaly analysis, and financial decision workflows.

C. Expense Forecasting

Expenditure forecasting uses Ridge Regression with strictly lagged temporal features. The model incorporates previous expenditure values, a three-month rolling average, and month information.

The Ridge objective is

$$\min_{\beta} \left[\sum_{i=1}^{n} (y_i - x_i^T \beta)^2 + \lambda \|\beta\|_2^2 \right], \tag{9}$$

where y_i is observed expenditure, x_i is the feature vector, β contains the learned coefficients, and λ controls regularization.

Temporal cross-validation uses TimeSeriesSplit. The implemented system reports a holdout MAE of INR 3,369.93 and provides a 90 percent uncertainty interval with the forecast.

D. Anomaly Detection

Isolation Forest is used for unsupervised detection of unusual expenditure observations. The implementation uses an empirical contamination parameter of 0.05.

The resulting signal indicates that an observation differs from the learned expenditure distribution; it is not treated as a definitive fraud classification. Contextual interpretation is therefore performed by the decision-support layer.

E. Regulatory Retrieval

FinMate uses RAG to provide financial and regulatory context to the language-model layer. The knowledge corpus contains guidance associated with RBI, SEBI, and FPSB India.

The implemented retrieval approach uses TF-IDF representations and cosine similarity. For query vector q and document vector d,

$$\cos(q, d) = \frac{q \cdot d}{\|q\| \|d\|}. \tag{10}$$

Documents are ranked according to similarity, and the highest-ranked entries are supplied as contextual evidence.

The implemented retrieval system reports a Hit Rate@3 of 93.3 percent and an average retrieval latency of 0.95 ms. This metric evaluates retrieval performance and does not by itself establish the correctness of the final language-model response.

F. Regulatory Grounding

The RAG layer establishes an evidence boundary between a user's financial question and language generation:

$$Q \rightarrow \text{Retrieve}(Q, K) \rightarrow K_r \rightarrow \text{Generate}(Q, K_r), \tag{11}$$

where Q is the query, K is the knowledge corpus, and K_r is the retrieved context.

This allows retrieved financial guidance to be distinguished from free-form language generation and provides a basis for displaying supporting regulatory information with financial explanations.

G. Large Language Model Abstraction

FinMate uses a common `BaseLLMProvider` interface so that the orchestration layer is not tied to a single model provider. The implementation includes:

- `MockLLMProvider` for deterministic testing;
- `OpenAILLMProvider` for OpenAI-based models; and
- `GeminiLLMProvider` for Google Gemini models.

The model receives structured information produced by backend services. Depending on the request, this may contain transaction summaries, financial calculations, forecasts, anomaly signals, scenario results, specialist outputs, and retrieved regulatory context.

H. Separation of Computation and Generation

A central architectural principle is that the language model is not the authoritative source for monetary values.

Financial calculations are performed by deterministic services, while predictive values originate from the corresponding machine learning models. These structured results are passed to the LLM for explanation and synthesis.

The conceptual flow is

$$D_{\text{financial}} \rightarrow C_{\text{deterministic}} \rightarrow E_{\text{structured}} \rightarrow L_{\text{narrative}}, \tag{12}$$

where $D_{\text{financial}}$ represents financial data, $C_{\text{deterministic}}$ represents authoritative computation, $E_{\text{structured}}$ represents structured evidence, and $L_{\text{narrative}}$ represents the generated explanation.

This separation reduces the possibility of treating generated text as the authoritative source of financial quantities.

I. Multi-Agent Decision Synthesis

The LLM layer is integrated with the multi-agent architecture through the centralized orchestrator. The orchestrator selects the specialist agents required for a query, including transaction, budget, goal, and investment specialists.

Their structured outputs are combined by the Financial Decision Agent. Not every query requires every specialist; the execution route depends on the detected intent and available evidence.

The decision layer can represent alternatives as a set

$$D = \{O_A, O_B, O_C\}, \tag{13}$$

where each O_i represents a possible scenario together with its financial impact, assumptions, and trade-offs.

This approach supports decision assistance without requiring the agent to autonomously execute consequential financial actions.

J. AI Governance Boundary

The AI subsystem operates within explicit execution constraints. The orchestrator limits execution to eight agent steps, twelve tool calls, and three agent handoffs. Tool access is additionally restricted through agent-specific allowlists.

FinMate also includes prompt-injection detection before governed agent execution. Inputs containing patterns associated with attempts to bypass system constraints, manipulate tool execution, or override governance are rejected before privileged tools can be invoked.

These controls establish a boundary between untrusted natural-language input and financial operations.

K. Integrated AI Decision Flow

For a typical financial query, the integrated process is:

1) The user submits a financial question or scenario.
2) The API validates the request.
3) The orchestrator identifies the relevant financial domain.
4) Specialist agents obtain permitted financial information.
5) Machine learning services provide analytical outputs when applicable.
6) The retrieval layer supplies relevant regulatory context.
7) Deterministic financial services calculate authoritative monetary values.
8) The Financial Decision Agent synthesizes the structured evidence.
9) The LLM generates a natural-language explanation.
10) Operations requiring persistent modification are routed to human approval rather than committed automatically.

The resulting pipeline combines predictive analysis, regulatory grounding, deterministic financial computation, and conversational interaction while maintaining explicit execution boundaries.

VII. Security and Privacy Architecture

FinMate processes personal financial information and therefore incorporates security and privacy controls across data ingestion, API access, agent execution, language-model interaction, and persistent state mutation. The architecture uses multiple independent controls rather than relying on a single security mechanism.

A. Security Design Principles

The security architecture follows four primary principles:

1) minimize exposure of sensitive financial information;
2) restrict agent capabilities through explicit permissions;
3) separate analysis and simulation from persistent state mutation;
4) require explicit human authorization for state-changing operations.

These principles are implemented across the application and data flow.

B. Human-in-the-Loop State Governance

FinMate distinguishes between read-only, simulation, and mutating operations. Read-only operations retrieve or analyze existing information, while simulation operations execute temporary calculations without modifying persistent records.

Operations that modify persistent financial state are placed into a pending approval state. The approval record expires after 15 minutes and requires explicit user confirmation before the proposed operation is committed.

The resulting transition is

$$P \rightarrow \begin{cases} C, & \text{explicit approval}, \\ R, & \text{rejection}, \\ E, & \text{expiration}, \end{cases} \tag{14}$$

where P, C, R, and E represent pending, committed, rejected, and expired states respectively.

This prevents an agent from silently modifying the user's persistent financial ledger.

C. Governed Agent Execution

FinMate provides fifteen governed domain tools with structured input and output contracts. Agent-specific allowlists restrict the tools available to each specialist.

The orchestrator additionally enforces maximum execution limits of eight agent steps, twelve tool calls, and three agent handoffs. These restrictions reduce uncontrolled execution and limit the capabilities available to any individual agent.

Tenant context is injected into domain operations so that tools operate within the authenticated financial context rather than accepting an arbitrary user identifier supplied by an agent.

D. Prompt Injection Protection

Natural-language input is inspected before governed agent execution. FinMate includes a prompt-injection detection mechanism intended to identify instructions that attempt to override system constraints, manipulate tool execution, or bypass agent governance.

Detected adversarial instructions are rejected before privileged financial tools can be invoked. This creates a boundary between untrusted natural-language input and operations capable of accessing or proposing changes to financial state.

E. Financial Computation Boundary

Financial calculations are isolated from language-model generation. Monetary operations use Python `Decimal` with `ROUND_HALF_UP`, while persistent monetary values use `NUMERIC(15,2)`.

The language model therefore receives structured financial results rather than becoming the authoritative source of monetary values. This separation allows numerical outputs to originate from deterministic backend services and analytical models while the LLM provides explanation and synthesis.

F. Android SMS Privacy

The FinMateSMS client is designed to minimize transmission of sensitive message content. Authorized bank messages are filtered and parsed locally on the Android device.

A sender whitelist restricts processing to recognized financial institutions, while OTP and secret-PIN patterns are filtered before transaction extraction. Eligible messages are parsed into structured transaction information and stored locally using AndroidX Room.

The raw SMS body is not transmitted to the backend.

A SHA-256 source reference and local uniqueness constraint provide duplicate protection before synchronization. The backend performs an independent duplicate check after receiving the structured transaction payload.

G. Secure Data Transmission and Validation

Structured transaction information is synchronized through the versioned FastAPI API. Retrofit and OkHttp provide the Android networking layer, while Pydantic schemas validate incoming backend payloads.

The backend performs duplicate verification before atomically inserting a new transaction. Since the synchronization payload contains structured transaction information rather than the original SMS body, the ingestion pipeline does not require transmission of raw SMS content.

H. Data Lineage and Auditability

FinMate retains transaction source type, source reference, import identifier, and data-quality metadata. Agent execution is additionally represented through task, approval, and tool-call records.

These records provide traceability for financial data entering the system and for governed operations performed over that data.

I. Layered Protection

The resulting security model combines local SMS processing, sensitive-message filtering, source hashing, duplicate prevention, structured API validation, tenant isolation, tool allowlists, bounded agent execution, prompt-injection detection, deterministic financial computation, and human approval.

These controls operate at different architectural boundaries and together support privacy protection and controlled access to persistent financial state.

VIII. Experimental Evaluation and Results

The implemented FinMate system was evaluated across machine learning, regulatory retrieval, multi-agent behavior, automated software testing, and frontend build validation. The reported results assess both individual analytical components and implementation-level system behavior.

TABLE I
TRANSACTION CLASSIFICATION RESULTS

Metric	Result
Accuracy	98.4%
Macro F1-score	0.9849
Categories	10
Classifier	LinearSVC
Features	TF-IDF

TABLE II
EXPENSE FORECASTING RESULT

Metric	Result
Model	Ridge Regression
Validation	TimeSeriesSplit
Holdout MAE	INR 3,369.93
Forecast interval	90%

A. Evaluation Dimensions

The evaluation considers:

1) transaction classification;
2) expenditure forecasting;
3) regulatory retrieval;
4) multi-agent benchmark performance;
5) automated software tests; and
6) production frontend build validation.

The reported metrics are implementation results and should be interpreted in the context of the datasets, benchmark scenarios, and evaluation procedures used during development.

B. Transaction Classification

The transaction classification subsystem uses TF-IDF features with a LinearSVC classifier across ten financial categories.

The implemented system reports 98.4 percent accuracy and a macro F1-score of 0.9849.

The macro F1-score gives equal importance to the defined transaction categories rather than weighting the result only by category frequency.

C. Expense Forecasting

The expenditure forecasting component uses Ridge Regression with lagged temporal features and TimeSeriesSplit during model development.

The implemented system reports a holdout Mean Absolute Error (MAE) of INR 3,369.93 and produces a 90 percent uncertainty interval.

The reported MAE should be interpreted together with the characteristics of the dataset used during development.

D. Regulatory Retrieval

The RAG subsystem was evaluated using retrieval hit rate and latency. The implemented system reports a Hit Rate@3 of 93.3 percent and an average retrieval latency of 0.95 ms.

This evaluation measures the ability of the retrieval subsystem to return relevant knowledge entries. It does not

TABLE III
REGULATORY RETRIEVAL RESULTS

Metric	Result
Hit Rate@3	93.3%
Average latency	0.95 ms
Similarity	Cosine similarity
Knowledge sources	RBI, SEBI, FPSB India

TABLE IV
MULTI-AGENT BENCHMARK

Metric	Result
Benchmark tasks	12
Successful tasks	12
Reported success rate	100%
Numerical consistency	Passed
Prompt-injection scenarios	Passed

independently measure the correctness of the final language-model response.

E. Multi-Agent Benchmark

The project benchmark contains twelve standard financial agent tasks. The implemented system completed all twelve reported scenarios.

The benchmark includes specialist requests, cross-domain workflows, approval-related behavior, numerical consistency checks, and prompt-injection scenarios.

F. Automated Software Testing

The project reports 102 automated tests passing out of 102 executed tests.

The test suite covers financial calculations, transaction classification, forecasting, anomaly detection, RAG retrieval, multi-agent orchestration, approval governance, tool registry behavior, CSV processing, data-quality rules, decision simulation, language-model guardrails, and REST API integration.

G. Frontend Build Validation

The production frontend uses TypeScript and Vite. The project reports a successful production build with zero errors and zero warnings across 1,892 transformed modules.

This provides an implementation-level validation of the frontend build in addition to the automated backend test suite.

H. Overall Results

The results demonstrate successful implementation and internal validation of the corresponding components. They should not be interpreted as evidence of universal financial prediction or decision-making performance because model results depend on the underlying datasets, evaluation procedures, and deployment conditions.

TABLE V
AUTOMATED TEST RESULTS

Metric	Result
Total tests	102
Passed	102
Failed	0
Reported pass rate	100%

TABLE VI
OVERALL FINMATE EVALUATION RESULTS

Evaluation Component	Result
Transaction accuracy	98.4%
Transaction macro F1	0.9849
Forecast holdout MAE	INR 3,369.93
RAG Hit Rate@3	93.3%
RAG latency	0.95 ms
Agent benchmark	12/12
Automated tests	102/102
Frontend build errors	0
Frontend build warnings	0

I. Evaluation Limitations

The current reported metrics do not provide a complete external generalization study. In particular, broader evaluation would benefit from documenting dataset size and temporal coverage, comparing against appropriate baseline models, and testing across more diverse transaction patterns.

Similarly, the RAG result measures retrieval rather than end-to-end generation correctness, while the twelve-task agent benchmark provides implementation validation but is not sufficient to characterize general agent behavior.

These limitations are considered when interpreting the reported results.

IX. DISCUSSION AND LIMITATIONS

The implementation of FinMate demonstrates how deterministic financial services, machine learning, retrieval-augmented generation, and multi-agent orchestration can be combined within a personal financial decision-support system. The principal design choice is to assign different responsibilities to different computational components rather than relying on a language model for all financial processing.

A. Integrated Intelligence

Machine learning models perform specialized analytical tasks, including transaction classification, expenditure forecasting, and anomaly detection. The deterministic financial engine calculates authoritative monetary quantities, while the RAG subsystem supplies domain-specific knowledge. The language model then operates primarily as an explanation and synthesis layer.

This separation provides a clearer computational boundary than treating the language model as the sole source of financial analysis.

B. Governed Multi-Agent Processing

The multi-agent architecture provides specialization across transaction, budget, goal, and investment domains. The centralized orchestrator coordinates these specialists while constrained tools and execution limits restrict their capabilities.

This architecture supports cross-domain financial queries but also introduces additional complexity, including routing errors, tool misuse, and inconsistent intermediate outputs. FinMate addresses these risks through structured schemas, bounded execution, tool allowlists, and human approval for state-changing operations.

These controls reduce the scope of agent actions but do not eliminate all risks associated with agentic systems.

C. Regulatory Grounding

The RAG subsystem provides an evidence layer for financial and regulatory questions. This is useful when a response depends on information outside the user's transaction history.

However, retrieval quality depends on the completeness and maintenance of the underlying knowledge corpus. A retrieved document can also be contextually similar without completely answering the user's question. Therefore, retrieval performance and final response correctness should be evaluated separately.

D. Privacy-Oriented Ingestion

The Android ingestion architecture reduces privacy exposure by performing SMS filtering and transaction extraction locally. The raw SMS body is not transmitted to the backend.

Sender filtering, sensitive-message exclusion, local storage, hashing, and duplicate prevention provide additional controls. Nevertheless, this approach depends on Android permissions and on the structure of bank-generated transaction messages. Changes in sender identifiers or SMS formats may require corresponding parser updates.

E. Human Agency

FinMate is intentionally designed as a decision-support system rather than an autonomous financial actor. Read-only analysis can execute automatically, while what-if scenarios remain isolated from persistent records. Operations that modify persistent state require explicit human approval.

This separation allows the system to assist financial decision making without independently executing consequential financial operations.

F. Evaluation Interpretation

The reported implementation results include 98.4 percent transaction classification accuracy, 0.9849 macro F1, 93.3 percent RAG Hit Rate@3, 102 out of 102 automated tests, and 12 out of 12 agent benchmark tasks.

These results demonstrate successful internal validation of the corresponding components. They should not be interpreted as evidence that the same performance will necessarily be obtained for different users, datasets, financial institutions, or deployment environments.

G. Limitations

Several limitations remain. First, the reported machine learning metrics depend on the datasets used during development and evaluation. Broader datasets are required to assess external generalization.

Second, the forecasting result is represented by a single reported holdout MAE. Additional baselines and evaluation periods would provide a stronger comparison.

Third, the RAG evaluation measures retrieval quality rather than complete end-to-end correctness of generated explanations.

Fourth, the multi-agent benchmark contains twelve scenarios. Larger and more diverse benchmark suites are required for broader evaluation of agent behavior.

Finally, FinMate does not independently execute financial transactions, fund transfers, or investment orders. Its purpose is to support human decision making while retaining explicit control over consequential state changes.

H. Deployment Implications

The architecture demonstrates the value of separating prediction, deterministic computation, retrieval, generation, and state mutation into distinct system boundaries. This separation allows individual components to be evaluated and improved independently while retaining explicit security and governance controls.

The architecture also provides a foundation for future experimentation with larger datasets, alternative predictive models, improved retrieval strategies, and additional language-model providers without replacing the core financial computation and governance mechanisms.

X. Future Work

Several directions remain for extending FinMate while preserving its deterministic financial computation and human-governed architecture.

A. Predictive and Retrieval Improvements

Future work can evaluate additional expenditure forecasting methods, including gradient-boosting and probabilistic models, across more diverse financial histories. The retrieval subsystem can similarly be extended with dense embeddings, hybrid retrieval, reranking, and versioned regulatory documents while retaining authoritative-source grounding.

B. Personalization and Explainability

Future versions can provide greater personalization based on longer-term financial behavior and user preferences. Such personalization should remain bounded by the deterministic financial engine and governance layer.

Decision provenance is another important direction. Future versions can link generated explanations to the underlying calculations, model outputs, retrieved sources, and agent actions so that users can inspect how a financial insight was produced.

C. Multi-Agent and Privacy Extensions

Future research can investigate adaptive task decomposition, cross-agent verification, and confidence estimation to determine whether more sophisticated coordination improves accuracy, reliability, or execution efficiency.

The Android ingestion pipeline can also be strengthened through additional on-device processing, improved sender classification, encrypted local storage, and broader privacy auditing across the complete data lifecycle.

D. Large-Scale Evaluation and Deployment

A larger evaluation should include diverse datasets, longer-term usage, user studies, additional baseline models, and concurrent system workloads. Measures such as decision comprehension, interaction effort, latency, resource consumption, and failure recovery would provide a broader assessment of system performance.

Deployment-oriented work can additionally introduce structured telemetry, distributed tracing, model monitoring, and retrieval monitoring. These extensions would support larger-scale operation without weakening the separation between financial computation, AI reasoning, data storage, and governed state mutation.

XI. Conclusion

This paper presented FinMate, a personal financial decision-support system that integrates deterministic financial computation, machine learning, retrieval-augmented generation, multi-agent orchestration, and privacy-focused transaction ingestion.

The central architectural principle is the separation of financial computation from language generation. Transaction analysis, expenditure forecasting, anomaly detection, regulatory retrieval, and deterministic financial calculations produce structured evidence, while the language model provides natural-language explanation and synthesis. State-changing operations are additionally controlled through governed tools and explicit human approval.

The implemented system reports 98.4 percent transaction classification accuracy with a macro F1-score of 0.9849, a 93.3 percent RAG Hit Rate@3, a forecasting holdout MAE of INR 3,369.93, and successful completion of 12 out of 12 agent benchmark tasks. The project also reports 102 out of 102 automated tests passing and a frontend production build with zero errors and zero warnings.

These results demonstrate the feasibility of combining conversational AI with deterministic financial computation, domain-grounded retrieval, machine learning, privacy-oriented ingestion, and human governance in a single financial decision-support architecture. FinMate is therefore designed to assist users with financial analysis while keeping numerical correctness, privacy controls, and authority over persistent financial changes outside the language model itself.

References

[1] S. Vaduka, S. C. Reddy, H. V. Arikathota, L. Konchada, P. Chakraborty, and A. Bandyopadhyay, "Optimizing personal finance management through ai-driven decision support systems," in *2024 IEEE Region 10 Symposium (TENSYMP)*, 2024.

[2] N. Shailendra, C. Parvathi, S. Gupta, and G. Ravikumar, "Money map: The personal finance tracker," in *2025 International Conference on Knowledge Engineering and Communication Systems (ICKECS)*, 2025.

[3] H. N. Lakshmi, C. V. L. Narayana, J. Krishna, A. Vishalakshi, S. Swama, and B. V. Sivaiah, "Ai-powered financial consultant for secure and intelligent personal finance management," in *2026 IEEE International Conference on Emerging Computing and Intelligent Technologies (ICoECIT)*, 2026.

[4] A. Alhenshiri, R. Shgaibe, and E. Abied, "An intelligent system for semantic classification of personal expenses from transaction descriptions," *The International Journal of Engineering & Information Technology*, vol. 14, no. 2, pp. 146–152, 2026.

[5] "Online transaction anomaly detection model for credit card usage using machine learning classifiers," in *2023 International Conference on Emerging Smart Computing and Informatics (ESCI)*, 2023.

[6] "Hybrid isolation forest-based machine learning for fraud detection in digital transactions," in *2026 6th International Conference on Intelligent Technologies (CONIT)*, 2026.

[7] Z. Yang, B. Biibosunov, S. Biibosunova, G. Kalmanbetova, and X. Wang, "A deep learning framework for automated transaction classification and budget prediction in personal financial management," *Discover Artificial Intelligence*, vol. 6, 2026.

[8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, S. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *arXiv preprint arXiv:2005.11401*, 2020.

[9] "A survey of retrieval-augmented generation (rag) for large language models," in *2025 International Conference on Trustworthy Big Data and Artificial Intelligence (ICTBAI)*, 2025.

[10] D. Maldonado, E. Cruz, J. Abad Torres, and P. J. Cruz, "Multi-agent systems: A survey about its components, framework and workflow," *IEEE Access*, vol. 12, pp. 80 950–80 975, 2024.

[11] Y. Wang, Y. Pan, Z. Su, Y. Deng, Q. Zhao, L. Du, T. H. Luan, J. Kang, and D. Niyato, "Large model-based agents: State-of-the-art, cooperation paradigms, security and privacy, and future trends," *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 1906–1949, 2026.

[12] D. Lee and H. Kim, "A survey on llm agents: Architecture, applications, and challenges," in *2026 40th International Conference on Information Networking (ICOIN)*, 2026, pp. 979–982.